

Docker Best Practices

Key guidelines for secure and portable Docker Images

ALLIE Arthur, Data Engineer
03 11 2025

Agenda

- What is Docker
- Difference between Image / Container
- Docker Best Practices in a nutshell
 - 1. Use official & verified images as base image
 - 2. Use specific image version
 - 3. Use small-size official images
 - 4. Use .dockerignore to exclude files and folders
 - 5. Optimizing image layers caching
 - 5'. Layer squashing
 - 6. Make use of “multi-stage builds” Dockerfile
 - 7. Use the least privileged user
 - 7'. Rootless Container
 - 8. Scan your images for vulnerabilities
 - 9. Using mounted volumes
 - 9'. The --reload tag to iterate quickly on app
 - 10. Add an Healthcheck
 - 11. Using ENTRYPOINT with CMD
 - 12. Mount sources into a volume→RUN layer

Docker best practices

- What is Docker ?

→ *A lightweight and portable containerization platform.*

→ **Definition :**

Docker allows you to run an application inside an **isolated container**, with all its **dependencies** (*libraries, environment variables, system packages, etc.*), in an **isolated, reproducible** and **portable** environment, ensuring identical behavior across all machines.

→ **Isolation:** Each container runs like a mini operating system, independent of the host (Container = lightweight OS).

→ **Reproducibility:** The same container runs identically on any machine (Linux, Windows, local, remote server, or cloud).

→ **Lightweight:** Containers share the host OS kernel → much lighter and faster than VMs.

→ **Scalability:** Easy to deploy and orchestrate (Docker Compose, Kubernetes, etc.).

Summary: → *“If it works on my machine, it will work everywhere.”*

Docker best practices

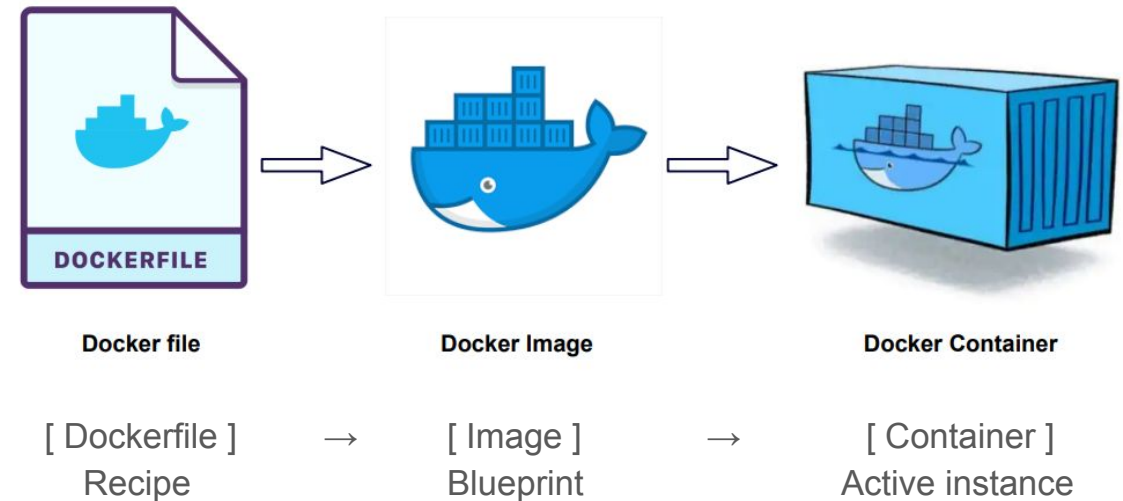
- Difference between Image / Container

Docker Image:

- This is the **blueprint** of an environment.
- Contains:
 - Base system (e.g., Debian, Ubuntu, Python).
 - Installed dependencies and libraries.
 - Application code and its configuration.
- **Immutable**: *once built, it does not change.*
- **Docker build**

Docker container:

- **The currently running instance** of an image.
- Can be started, stopped, deleted, or recreated from the same image.
- **Mutable**: Can write data (persistent via volumes).
- **Docker run**



Docker best practices

- Docker Best Practices in a nutshell

→ Two major axes:

→ Reducing image size

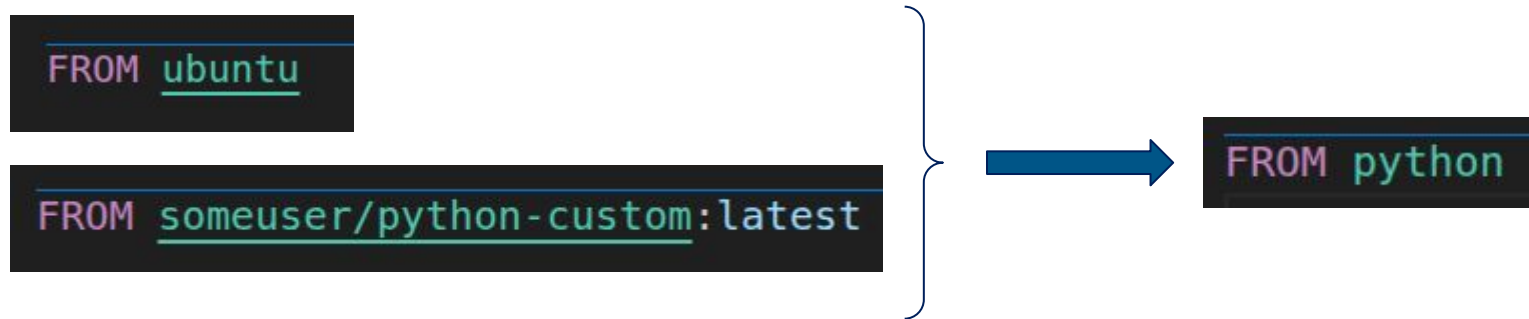
→ Reducing surface of attack



→ *By reducing the image size ⇒ reduce the surface of attack.*

1. Use official and verified Docker images as base image

- Instead of taking a **base operating image system** → use the **official python image**.



→ Avoid install all of the dependencies by yourself.

- Start with an **official image maintained by Docker** or the publisher (e.g., Python).

→ Guarantee of security updates and compatibility with Docker Hub.

2. Use specific image version

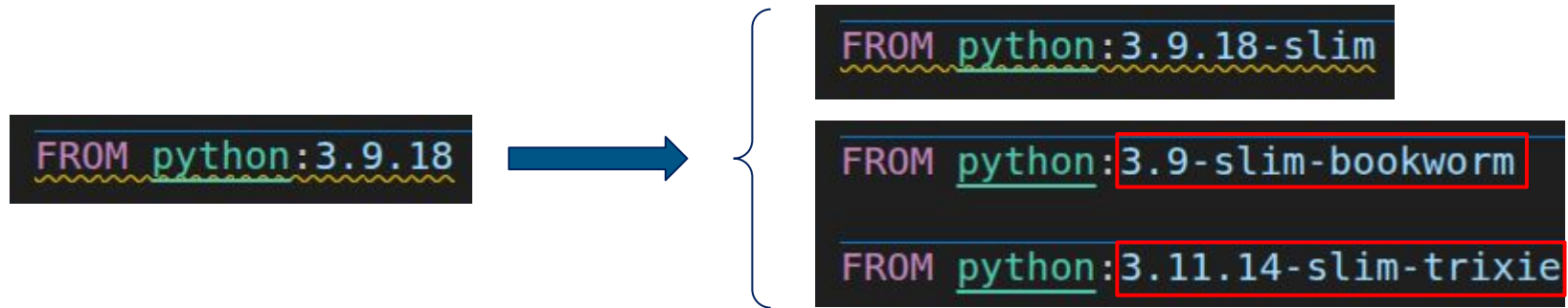
- Use a specific image version in the format *python:3.x.y*
 - Python **3** (*major*) . **x** (*minor*) . **y** (*patch*)
 - e.g. python:3.9.18



→ Avoid that a change to “latest” breaks the build during an update.

3. Use small-size official images

- Choose an optimized base with the most specific possible tag.



- Goal in terms of **size**: reducing the **size** of the final image.
- Goal in terms of **security**: reducing the **surface** of attack.
- Typical reduction: from several 100MB to a few dozen.
- Warning: if using Alpine (`python:3.*.*-alpine`), sometimes breaks C/C++ builds (numpy, torch, etc.).
- Choose a maintained and flawless image (**no high vulnerabilities**).

4. Use .dockerignore to exclude files and folders

- Use *.dockerignore* to explicitly exclude files and folders.

→ We don't need everything inside the Docker image (README, auto-generated folders, etc.)

→ Keeping dependencies files:

→ required for *docker build* step (.json, .xml, **building tools**: *jdk*, *sbt*, *maven*, etc.).

→ Goal in terms of **size**: reducing the **size** of the final image.

→ Goal in terms of **security**:

→ Reducing the **surface** of attack.

→ Prevent unintended **sensitive data exposure**: secrets, keys, logs, etc.

- The process:

→ Create **.dockerignore** file in the root directory of the project.

→ List files and folders to ignore.

```
# Exclude unnecessary or sensitive files
__pycache__/
*.pyc
*.log
*.env
data/
logs/
python_modules/

# ignore .git and .cache folders
.git
.cache

# ignore all markdown files
*.md

# ignore sensitive files
secrets/
private.key
settings.json
```

5. Optimizing image layers caching

- The rule:

→ Each layer = **immutable** & contains only **changes from the previous layer** (not from the following one).

⇒ A **change / rebuild in one layer** causes a **rebuild** in **ALL** the **following layers**.

- In that example, the **application code** could change a lot.

→ Should be as low as possible in the Dockerfile.

→ If the code changes:

→ **Only the last layer** is rebuilt,

→ **Not the entire Python environment** (*requirements.txt*).

```
FROM python:3.11-alpine3.21 (last pushed 6 days ago)

WORKDIR /app

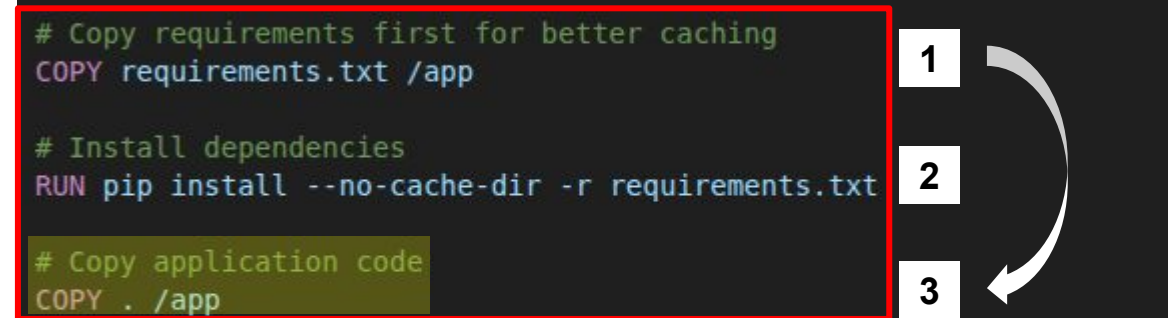
# Copy requirements first for better caching
COPY requirements.txt /app

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY . /app

# Expose the port FastAPI runs on
EXPOSE 8000

# Command to run the application
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```



5. Optimizing image layers caching

- Order Dockerfile commands *from least to most frequently changing* (to benefit from **image layer caching**).

```
FROM python:3.11-alpine3.21
WORKDIR /app

# Copy application code
COPY . /app

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Expose the port FastAPI runs on
EXPOSE 8000

# Command to run the application
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

← This is a layer

← This is another layer

1

2



```
FROM python:3.11-alpine3.21 (last pushed 6 days ago)
WORKDIR /app

# Copy requirements first for better caching
COPY requirements.txt /app

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY . /app

# Expose the port FastAPI runs on
EXPOSE 8000

# Command to run the application
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

1

2

3

5'. Layer squashing

- Layer squashing (*reduce the number of layers*).

```
RUN apt-get update
RUN apt-get install -y --no-install-recommends python3 python3-dev \
python3-pip build-essential \
wget curl git
RUN rm -rf /var/lib/apt/lists/*
RUN ln -s /usr/bin/python3 /usr/bin/python
```



```
RUN apt-get update \
&& apt-get install -y --no-install-recommends \
python3 \
python3-dev \
python3-pip \
build-essential \
wget \
curl \
git \
&& rm -rf /var/lib/apt/lists/* \
&& ln -s /usr/bin/python3 /usr/bin/python
```

- The rule:

→ Each layer = **immutable** & contains **only changes from the previous layer**.

⇒ **Separate RUN** commands **create new layers** (preserving files from earlier layers).

⇒ Deleted files marked as “not accessible” / still **consume space** in image.

→ Docker **can't** remove data **from previous layers** due to their **immutability**.

5'. Layer squashing

- Group **RUNs together** when possible.

⇒ Avoid **multiplying immutable layers**.

⇒ (Each **RUN** creates a **new layer**).

→ Merging them **reduces size & number of layers**.

→ Gain in **size**:

→ Reduce image final size (*each layer contains writings*).

→ When the layer is committed → Only final state with the cleaned up files.

→ Not any intermediate state with the extra files.

→ Gain in **readability**

- The only downside:

Single RUN ⇒ no immutability ⇒ any change ⇒ **can't benefit** from **image layer caching** anymore.



```
RUN apt-get update \  
    && apt-get install -y --no-install-recommends \  
        python3 \  
        python3-dev \  
        python3-pip \  
        build-essential \  
        wget \  
        curl \  
        git \  
    && rm -rf /var/lib/apt/lists/* \  
    && ln -s /usr/bin/python3 /usr/bin/python
```

6. Make use of “multi-stage builds” Dockerfile

- **“Multi-stage”** builds allows to **compile** in a **complete** (first) environment, & then to **run** in a **minimal** and **secure** (second) environment.
- The process :
 - Compile/install dependencies - *everything needed to build the application* - in a **“builder”** stage.
 - Compilers (*gcc, make*)
 - Development libraries (*python3-dev, build-essential*)
 - Managers (*pip, npm, git*)
 - Copy only the necessary files - *only what's needed to run the application* - into a **“runtime”** stage.
 - Avoid embedding build tools into the final image (*gcc, cargo, jdk, sbt, maven, etc.*).
 - **Exclude** entire **compilation** environment / retain only **dependencies required for execution**.

6. Make use of “multi-stage builds” Dockerfile

- Goal:
 - Drastic final image **size reduction**:
 - Compile, install, download all the heavy components (dependencies, build tools, etc.) → **temporary** image (build stage).
 - Copy only what's strictly **necessary** (*to run the app*) into the **final** image (runtime stage).
 - e.g. from 8GB → 2GB
 - *The code is transformed into ready-to-use **artifacts** (binaries, wheels, libs, etc.).*
 - Better **security & portability**:
 - Reducing the **surface** of attack: *fewer layers ⇒ fewer vulnerabilities.*
 - Prevent unintended **sensitive data leaks**: *secrets, SSH keys, temporary cache, etc.*
 - Easier to **distribute** and faster to **start**.
 - *Runtime stage only retrieves produced artifacts (executables, libraries, template files, etc.).*
 - Cleaner, more maintainable.

6. Make use of “multi-stage builds” Dockerfile

```
# =====  
# Simple stage  
# =====  
FROM python:3.11-slim-bookworm (last pushed 1 hour ago)  
  
WORKDIR /app  
  
RUN apt-get update && apt-get install -y build-essential  
COPY requirements.txt /app  
RUN pip install -r requirements.txt  
  
COPY . /app  
  
EXPOSE 8000  
  
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```



```
# =====  
# Stage 1 – Builder Stage (compilation, install, etc.)  
# =====  
FROM python:3.11-slim-bookworm AS builder (last pushed 1 hour ago)  
  
WORKDIR /app  
  
RUN apt-get update && apt-get install -y build-essential  
COPY requirements.txt /app  
RUN pip install -r requirements.txt  
  
# =====  
# Stage 2 – Runtime Stage (lighter)  
# =====  
FROM python:3.11-slim-bookworm AS runtime (last pushed 1 hour ago)  
  
WORKDIR /app  
  
COPY --from=builder /usr/local/lib/python3.11/site-packages \ /usr/local/lib/python3.11/site-packages  
  
COPY . /app  
  
EXPOSE 8000  
  
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

7. Use the least privileged user

- Goal:
 - Prevent a process in the container from having **root privileges**.
 - Limit impact if compromised:
 - With root privilege: easier privilege escalation for an attacker,
(which can, from inside the container, escalate to root privileges on the host machine).
 - Reducing the “**surface**” of attack.
 - *The process now runs with a **standard user** (appuser).*
- → Already managed into Node-based or Python-based images for instance.

7. Use the least privileged user

```
FROM python:3.11-slim-bookworm (last pushed 10 hours ago)

WORKDIR /app

RUN apt-get update && apt-get install -y build-essential

COPY requirements.txt /app
RUN pip install -r requirements.txt
COPY . /app

EXPOSE 8000
CMD ["python", "main.py"]
```



```
FROM python:3.11-slim-bookworm (last pushed 1 day ago)

WORKDIR /app

RUN apt-get update && apt-get install -y build-essential

COPY requirements.txt /app
RUN pip install -r requirements.txt
COPY . /app

# Create a non-root user for security
RUN useradd -m -u 1001 appuser \
    && chown -R appuser:appuser /app
USER appuser

EXPOSE 8000
CMD ["python", "main.py"]
```

7'. Rootless Container

- Reminder:

→ With root privilege: easier privilege escalation for an attacker,
(which can, from inside the container, escalate to root privileges on the host machine).

- Goal:

→ Run the container without root privileges on the host side.

→ Better **security & isolation**.

→ **Alternative** or complement to *non-root USER*.



```
# Rootless installation
dockerd-rootless-setup tool.sh install
```

```
# Starting Docker rootless
systemctl --user start docker
```

```
# Verification
docker info | grep Rootless
```

```
# Typical output:
```

```
Rootless: true
```

```
docker run -it --rm python:3.9-alpine whoami
```

```
# -> root (in the container)
```

```
# -> but user 1000 on the host
```

7'. Rootless Container

- Other alternatives:
 - **Podman**: designed “**rootless**” from the **beginning**.
 - **Containerd** (**lightweight** engine for *Kubernetes*).



Classic *Docker*, root on the host

```
docker run my-image
```



Rootless execution with *Docker Rootless* mode or *Podman*

```
dockerd-rootless-setup.sh install
systemctl --user start docker
docker run my-image

# or with Podman:
podman run my-image
```

8. Scan your images for vulnerabilities

- Goal:
 - Identify *Common Vulnerability and Exposure* (CVEs).
 - The scanners automatically detect vulnerable packages in the Dockerfile layers.
 - Maintain compliance and security throughout the DevOps lifecycle.
- Tools: *Docker Scan, Trivy, Snypk, (Docker Hub), JFrog (CI tool on AForge).*

Docker Scan method (built-in)

`docker scan myimage`

With Trivy (more comprehensive)

`trivy image myimage`

With Snypk

`snypk container test myimage`

Confidentiality reasons



9. Using mounted volumes

- **Volumes** allow to **link a local folder** to the container **without copying** it into the image.

→ Ideal for separating **code** from **persistent data** (models, log files, caches, ...).

```
docker run -v $(pwd)/shared_data:/app/shared_data my-image:0.1.0
```

→ Avoids rebuilding the image after every local change (combined with the `--reload` tag).

→ Makes it easier to debug and iterate on your application in real time.

10. Add an *Healthcheck*

- Allows Docker to **automatically check the service status** at regular intervals.

→ Useful for orchestrators (Compose, Kubernetes, Swarm).

```
HEALTHCHECK --interval=30s --timeout=10s \  
  CMD curl -f http://localhost:8000/health || exit 1
```

→ The container is marked as “unhealthy” if the application stops responding.

9'. The `--reload` tag & a mounted volume to iterate quickly on app

- Before:

→ `docker build -t <image-name:tag>` → `docker run <image-name:tag>`

→ `docker stop <container-name>` → `docker rm <container-name>`

→ **Code changes**

→ `docker build -t <image-name:tag>` → `docker run <image-name:tag>`

→ `docker stop <container-name>` → `docker rm <container-name>`

...



- After:

→ `docker build -t <image-name:tag>` → `docker run <image-name:tag>`

→ **Code changes**

→ *No re-build & no re-run*

```
docker run \  
  -v $(pwd):/app \  
  -e FASTAPI_ENV=dev \  
  my-image:0.1.0
```

```
# =====  
# Stage 2 - Runtime Stage: lightweight final image  
# =====  
FROM python:3.11.14-slim-trixie AS runtime  
  
# Environment & encoding + fix local and encoding issues + set home to /app  
ENV PYTHONDONTWRITEBYTECODE=1 \  
  PYTHONUNBUFFERED=1 \  
  DEBIAN_FRONTEND=noninteractive \  
  LANG=C.UTF-8 \  
  LC_ALL=C.UTF-8 \  
  PYTHONIOENCODING=utf-8 \  
  PYTHONPATH=/app:/opt/python \  
  HOME=/app \  
  FASTAPI_ENV=dev  
  
# ...  
# ...  
  
EXPOSE 5000  
  
HEALTHCHECK --interval=30s --timeout=10s --start-period=60s --retries=3 \  
  CMD curl -fsS http://localhost:${API_PORT:-5000}/health || exit 1  
  
# Command to run the application  
# CMD ["python3", "-m", "uvicorn", "main:app", "--host", "0.0.0.0", "--port", "5000"]  
CMD ["/bin/sh", "-c", "  
if [ \"$FASTAPI_ENV\" = \"dev\" ]; then \  
  python3 -m uvicorn main:app --host 0.0.0.0 --port 5000 --reload; \  
else \  
  python3 -m uvicorn main:app --host 0.0.0.0 --port 5000; \  
fi"]
```

11. Using ENTRYPOINT with CMD

- **ENTRYPOINT** defines the **main, non-overridable command**, while **CMD** can be overloaded at docker run.

```
ENTRYPOINT ["python3", "-m", "uvicorn", "main:app"]  
CMD ["--host", "0.0.0.0", "--port", "8000"]
```

→ Combining both: ENTRYPOINT fixes the binary, CMD provides the default arguments.

12. Mount sources into a volume during a RUN layer

- **Docker BuildKit**: → temporarily mount external file / folder **during the build**, without copying it into the image.

```
# syntax=docker/dockerfile:1.4  
FROM python:3.11.14-slim-trixie  
  
RUN --mount=type=bind,source=./archive.zip,target=/tmp/archive.zip \  
    apt-get update && apt-get install -y unzip && \  
    unzip /tmp/archive.zip -d /app && \  
    rm -rf /tmp/archive.zip
```

→ **archive.zip** never included in the image

(.zip temporarily mounted from host → **/tmp/archive.zip**).

→ Only available during the **RUN**.

→ At the end of the **RUN**, Docker “commits” the layer:

→ **/app** contents are **persisted** in the new layer,

→ **/tmp/archive.zip** & entire **--mount** configuration are lost.

→ Requires BuildKit to be enabled (*DOCKER_BUILDKIT=1 docker build ...*).

→ Unsupported by Docker Hub builds (classic CI/CD environment).